A Spatiotemporal Graph-of-Graphs Framework for Multivariate Time Series Classification

A Project Report

Submitted in Partial Fulfillment of the Requirements
For the Award of the Degree
of

Master of Technology
in
Information Technology

Submitted by:
Tushar Sharma (2024WNC-012)

Under the supervision of
Dr. Anjali



DEPT. OF INFORMATION TECHNOLOGY
Atal Bihari Vajpayee Indian Institute of Information Technology & Management
Morena Link Road, Gwalior-474015
Jan 2026 – May 2026

DEPT. OF INFORMATION TECHNOLOGY
ATAL BIHARI VAJPAYEE INDIAN INSTITUTE OF INFORMATION TECHNOLOGY & MANAGEMENT
Morena Link Road, Gwalior-474015

CANDIDATE'S DECLARATION

I, **Tushar Sharma (2024WNC-012)**, student of Master of Technology (Information Technology), hereby declare that the Project Report titled "A Spatiotemporal Graph-of-Graphs Framework for Multivariate Time Series Classification" which is submitted by me to the Department of Information Technology, ABV-IIITM, Gwalior in fulfilment of the requirement for awarding of the Master of Technology degree, is not copied from any source without proper citation. This work has not previously formed the basis for the award of any Degree, Diploma, Fellowship or other similar title or recognition.

Place: Gwalior, M.P.

Date: _____

Candidate's Signature
Tushar Sharma

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Place: Gwalior, M.P.

Date: _____

Signature of Supervisor
Dr. Anjali

ATAL BIHARI VAJPAYEE INDIAN INSTITUTE OF INFORMATION TECHNOLOGY & MANAGEMENT

Morena Link Road, Gwalior-474015

CERTIFICATE

I hereby certify that the Project titled "A Spatiotemporal Graph-of-Graphs Framework for Multivariate Time Series Classification", which is submitted by Tushar Sharma (2024WNC-012) for fulfilment of the requirements for awarding of the degree of Master of Technology (M.Tech.) is a record of the project work carried out by the student under my guidance & supervision. To the best of my knowledge, this work has not been submitted in any part or fulfilment for any Degree or Diploma to this University or elsewhere.

Place: Gwalior, M.P.
Date: _____

Dr. Anjali
(SUPERVISOR)
Assistant Professor
Department of Information Technology
ABV-IIITM Gwalior

Acknowledgements

I would like to express my sincere thanks to all those people who made this report possible. First and foremost, I would like to express my profound respect and gratitude to my supervisor, Dr. Anjali, who has been the guiding force behind this work. I am greatly indebted for her invaluable guidance, constant encouragement, and her valuable comments on my work. I am fortunate enough to have such an advisor who gave me the freedom to think independently and explore new ideas. More importantly, I would like to thank her for the patience she has shown in carefully reading and commenting on the manuscripts, and the countless revisions of this dissertation. Her commitment and dedication to research have been and will continue to be a constant source of inspiration for me. I am highly privileged to have got an opportunity to work with such a wonderful person.

Tushar Sharma

Abstract

Assigning class labels to multivariate time series simultaneous recordings from multiple sensors arises in cardiac diagnostics, seizure monitoring, industrial fault detection, gesture recognition, and traffic-flow analysis. The difficulty is not merely scale: three qualitatively distinct dependency structures must be resolved at once. Within each channel, relevant events unfold through time. Meaningful patterns frequently arise from the interactions between various channels rather than from any one channel alone. As the recording progresses, these inter-channel interactions may also change, making the underlying dynamics extremely non-stationary. Few strategies are able to address all three issues in a single framework, however existing methods usually succeed in addressing some parts of this issue.

This thesis proposes the **Spatiotemporal Graph-of-Graphs (ST-GoG) Framework**. A **Residual TCN with Squeeze-and-Excitation attention** extracts per-channel temporal features through five dilated causal convolutional blocks at dilation factors $d \in \{1, 2, 4, 8, 16\}$; an SE block then reassigns importance weights across feature maps based on their aggregate discriminative signal. A **Multi-Scale GCN** processes the TCN embeddings through three independent graph convolutional branches, each using chain-adjacency at a different radius ($r \in \{1, 3, 5\}$), with learnable softmax scalars controlling how the three outputs are blended. A **hierarchical GoG module** partitions each recording into temporal segments, applies an inner GCN within each segment, and uses an outer GCN to propagate context across segments.

Training uses MixUp augmentation ($\alpha = 0.2$), cosine annealing with warm restarts (T_0=50, T_{mult}=2), exponentially decaying stochastic depth, adaptive label smoothing, and seven-pass test-time augmentation. A learnable four-pathway softmax head combines all four feature streams before classification.

Evaluated on all 22 UEA datasets, ST-GoG achieves the highest mean test accuracy (0.743) and wins on more individual datasets than any single competing method. Gains are sharpest on small-training-set medical benchmarks and structured motion-capture tasks.

Keywords – Multivariate Time Series Classification, Graph Convolutional Networks, Temporal Convolutional Networks, Graph-of-Graphs, Spatiotemporal Learning, Hierarchical Pooling, Squeeze-and-Excitation, MixUp, Deep Learning.

List of Acronyms

List of Acronyms

Acronym	Full Form
AI	Artificial Intelligence
BN	Batch Normalization
CNN	Convolutional Neural Network
DTW	Dynamic Time Warping
ECG	Electrocardiogram
EEG	Electroencephalogram
FCN	Fully Convolutional Network
FDNet	Feature and Difference Network (Specific Model)
GAP	Global Average Pooling
GCN	Graph Convolutional Network
GFLOPs	Giga Floating-Point Operations Per Second
GNN	Graph Neural Network
GoG	Graph-of-Graphs
LSTM	Long Short-Term Memory
MLP	Multi-Layer Perceptron
MTSC	Multivariate Time Series Classification
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
ROCKET	Random Convolutional Kernel Transform
SAGoG	Similarity-Aware Graph-of-Graphs (Specific Model)
SE	Squeeze-and-Excitation
SGD	Stochastic Gradient Descent
Softmax	Soft-Maximum (Activation Function)
ST-GoG	Spatiotemporal Graph-of-Graphs (Proposed Model)
SVM	Support Vector Machine
TapNet	Time Series Attentional Prototypical Network
TCN	Temporal Convolutional Network
TD-GCN	Temporal Dynamic Graph Convolutional Network
TSC	Time Series Classification
TTA	Test-Time Augmentation
UEA	University of East Anglia (Time Series Archive)
WD	Weight Decay

Contents

Candidate's Declaration ii

Certificate iii

Acknowledgements iv

Abstract v

List of Acronyms vi

1 Introduction 1
- 1.1 Classical Approaches . . . 2
- 1.2 Deep Learning for TSC . . . 2
- 1.3 Multivariate Time Series Classification (MTSC) . . . 3
  - 1.3.1 Graph-Based Approaches to MTSC . . . 3
- 1.4 Motivation for the Proposed Work . . . 3
- 1.5 Contributions of This Work . . . 4
- 1.6 Report Organization . . . 4

2 Literature Review 5
- 2.1 Overview . . . 6
- 2.2 Distance-Based and Feature-Based Methods . . . 6
  - 2.2.1 Dynamic Time Warping and Elastic Measures . . . 6
  - 2.2.2 Interval and Shapelet Methods . . . 6
  - 2.2.3 Random Kernel Transforms . . . 6
- 2.3 Deep Learning Approaches . . . 6
  - 2.3.1 Convolutional Architectures . . . 6
  - 2.3.2 Recurrent Architectures . . . 7
  - 2.3.3 Temporal Convolutional Networks . . . 7
  - 2.3.4 Attention and Transformer Architectures . . . 7
- 2.4 Graph-Based Approaches . . . 7
  - 2.4.1 Graph Convolutional Networks . . . 7
  - 2.4.2 GCNs for Time Series . . . 8
  - 2.4.3 FDNet . . . 8
  - 2.4.4 SAGoG . . . 8
- 2.5 Comparative Review of FDNet and SAGoG . . . 9

2.6 Synthesis: Research Gaps . . . 9

3 Problem Statement and Objectives 10
3.1 Formal Problem Definition . . . 11
3.2 Mathematical Formulation of Graph-Based MTSC . . . 11
3.2.1 Channel Graph Construction . . . 11
3.2.2 GCN Message Passing . . . 11
3.3 Challenges in MTSC . . . 11
3.3.1 Heterogeneous Temporal Dynamics . . . 11
3.3.2 Adaptive Segmentation . . . 12
3.3.3 Scalability and Computational Constraints . . . 12
3.3.4 Overfitting on Small Datasets . . . 12
3.4 Identified Research Gap . . . 12
3.5 Research Objectives . . . 12

4 Methodology and Proposed Framework 14
4.1 Overview of the ST-GoG Framework . . . 15
4.2 Research Workflow . . . 15
4.2.1 Step 1: Data Preparation and Normalization . . . 15
4.2.2 Step 2: Chain Adjacency Construction . . . 15
4.2.3 Step 3: Temporal Encoding via Residual TCN with SE Attention . . . 15
4.2.4 Step 4: Multi-Scale Spatial Encoding via GCN . . . 16
4.2.5 Step 5: Hierarchical GoG Module . . . 16
4.2.6 Step 6: Pathway Fusion and Classification . . . 17
4.3 Training Strategy . . . 17
4.3.1 Loss Function . . . 17
4.3.2 MixUp Data Augmentation . . . 17
4.3.3 Cosine Annealing with Warm Restarts . . . 18
4.3.4 Stochastic Depth . . . 18
4.3.5 Test-Time Augmentation . . . 18
4.3.6 Optimizer and Regularization . . . 18
4.4 High-Level Architecture . . . 18
4.5 Design Considerations and Trade-offs . . . 19
4.5.1 Chain Adjacency vs. Cosine Similarity . . . 19
4.5.2 Adaptive Segment Count for GoG . . . 19
4.5.3 Learnable vs. Fixed Pathway Weights . . . 19

5 Implementation and Experimental Results 20
5.1 Implementation Environment . . . 21
5.2 Baseline Model Implementation . . . 21
5.2.1 Overview . . . 21
5.2.2 Temporal Encoder (TCN) – Baseline . . . 22
5.2.3 Spatial Encoder (GCN) – Baseline . . . 22

5.2.4 GoG Aggregation – Baseline . . . 23
5.2.5 Baseline Training Hyperparameters . . . 24
5.3 Improved Architecture (ST-GoG) . . . 24
5.3.1 Overview . . . 24
5.3.2 Residual TCN with Squeeze-and-Excitation . . . 25
5.3.3 Multi-Scale GCN . . . 27
5.3.4 Hierarchical GoG Module . . . 27
5.3.5 Pathway Fusion and Classification . . . 27
5.3.6 Improved Training Hyperparameters . . . 28
5.4 Dataset Description . . . 28
5.5 Baseline Experimental Results . . . 29
5.5.1 Baseline Results on Four Datasets . . . 29
5.6 Full Benchmark: Improved ST-GoG Architecture . . . 30
5.7 Category-Wise Ranking Analysis . . . 31
5.7.1 Category Definitions and Dataset Mapping . . . 31
5.7.2 Category-Wise Ranking Summary . . . 31
5.7.3 Motion and Activity Recognition . . . 32
5.7.4 Medical and Neurological . . . 32
5.7.5 Sensor and High-Dimensional . . . 32
5.7.6 Astronomy and Biology . . . 32
5.8 Ablation Study . . . 33
5.8.1 Learned Weight Analysis . . . 33
5.9 Discussion of Results . . . 33
5.9.1 Datasets Where ST-GoG Excels . . . 33
5.9.2 Datasets Where Competitors Are Stronger . . . 34
5.9.3 Win/Tie/Loss Summary . . . 34
5.10 Limitations and Future Improvements . . . 34

6 Conclusions and Future Scope **35**
6.1 Summary of Contributions . . . 36
6.2 Key Findings . . . 36
6.3 Future Research Directions . . . 36
6.3.1 Data-Driven Dynamic Adjacency . . . 36
6.3.2 Attention-Based GoG Pooling . . . 37
6.3.3 Scalable Instance Graph Approximation . . . 37
6.3.4 Temporal Adaptive Sampling . . . 37
6.3.5 Interpretability . . . 37
6.3.6 Pre-Training and Transfer Learning . . . 37
6.3.7 Real-Time and Edge Deployment . . . 37

A Additional Experimental Details **38**
A.1 Dataset Descriptions by Domain . . . 38
A.2 Full Hyperparameter Settings . . . 39

A.3 Ablation Study Design 39
A.4 Project Timeline 40

List of Figures

4.1 ST-GoG three-tier architecture overview. 19

5.1 Baseline TCN–GCN–GoG pipeline. 22
5.2 Improved ST-GoG end-to-end architecture. 25
5.3 Internal structure of the three core modules. 26
5.4 Average rank diagram across 22 UEA datasets. 31

List of Tables

2.1 Detailed comparison of FDNet, SAGoG, and the proposed ST-GoG framework. . 9

5.1 Implementation environment 21
5.2 Baseline model hyperparameters 24
5.3 Improved ST-GoG model hyperparameters 28
5.4 Summary of UEA datasets used in evaluation 29
5.5 Accuracy comparison: TCN–GCN–GoG baseline vs. SAGoG and ROCKET . . . 29
5.6 Classification accuracy across all 22 UEA datasets. 30
5.7 Domain category assignment of the 22 UEA benchmark datasets. 31
5.8 Category-wise models ranking summary 32
5.9 Ablation study on PEMS-SF. 33
5.10 Learned fusion weights of the full ST-GoG on PEMS-SF. 33
5.11 Win/tie/loss count for ST-GoG vs. each baseline across 22 datasets. 34

A.1 Full hyperparameter settings for both baseline and improved model 39
A.2 Proposed ablation study variants 40
A.3 Work Plan and Timeline 40

1

Introduction

Sensor networks, clinical instruments, and motion-capture systems all produce the same kind of raw output: a table of numbers in which each column records a measurement channel and each row corresponds to a moment in time. The scientific value of that table lies not in any individual reading but in the patterns that emerge across consecutive rows the periodic oscillation that characterises a healthy heartbeat, the transient amplitude spike that precedes a mechanical failure, the co-ordinated sweep of joint angles that distinguishes one gesture from another. Assigning a class label to such a recorded sequence deciding, from the pattern of measurements alone, which of several known categories the recording belongs to is the task of time series classification (TSC). It arises in nearly every domain that relies on continuous instrumentation, from healthcare and structural monitoring to robotics and activity recognition.

Considerable progress has been made on the univariate version of this problem, where a single channel is observed. The harder and less-solved variant arises when several channels are recorded simultaneously the setting known as **Multivariate Time Series Classification (MTSC)**. In the multivariate case the model must jointly address three interdependent challenges: detecting discriminative patterns *within* individual channels, characterising the structural dependency *between* channels, and tracking how that inter-channel dependency changes as the recording progresses. A model that handles only one or two of these challenges ignores part of the data that may most clearly distinguish the classes from one another. This combined temporal-and-structural problem is the central subject of this thesis.

1.1 Classical Approaches

The earliest successful TSC methods were distance-based. The **1-Nearest Neighbour classifier paired with Dynamic Time Warping (DTW)** [11] became the dominant baseline for over a decade. DTW is an elastic alignment measure that finds the optimal non-linear mapping between two sequences, allowing comparison even when they differ in speed or phase. Formally, given two sequences $\mathbf{a} = (a_1, \ldots, a_m)$ and $\mathbf{b} = (b_1, \ldots, b_n)$, DTW computes:

$$\mathrm{DTW}(\mathbf{a}, \mathbf{b}) = \min_{\mathcal{W}} \sum_{(i,j)\in\mathcal{W}} d(a_i, b_j),$$

where $\mathcal{W}$ is a warping path a sequence of index pairs (i, j) satisfying boundary, monotonicity, and continuity conditions. Despite its effectiveness on small-to-medium datasets, DTW scales as $O(mT)$ per pair, making it infeasible for large datasets or real-time applications.

Feature-based methods offered an alternative: extract a fixed-dimensional feature vector from each time series and train any off-the-shelf classifier. Common features include statistical moments (mean, variance, skewness), frequency-domain descriptors (FFT coefficients, spectral entropy), and symbolic representations (SAX [20], shapelets [19]). The **ROCKET** [6] framework pushed this paradigm to its extreme efficiency: a large bank of random convolutional kernels of varying lengths and dilations is applied to each series, and the maximum value and proportion of positive values of each convolution output are extracted as features. A simple linear classifier (ridge regression) trained on these features achieves near state-of-the-art accuracy at a fraction of the computational cost of ensemble or deep learning methods.

1.2 Deep Learning for TSC

Convolutional Neural Networks (CNNs) were among the first deep architectures applied successfully to TSC. A **Fully Convolutional Network (FCN)** [15] consisting of stacked 1D convolution, batch normalization, and ReLU layers with a Global Average Pooling (GAP) head achieved competitive accuracy without any handcrafted features. **ResNet** [17] variants introduced skip connections to enable deeper architectures. **InceptionTime** [8], inspired by Google's Inception module, applied multiple convolutional filters of different kernel sizes in parallel and fused their outputs, capturing patterns at multiple temporal scales simultaneously.

Recurrent architectures, particularly **Long Short-Term Memory (LSTM)** networks and their variants, were another important direction. An LSTM maintains a hidden state that evolves over time, allowing it to capture long-range dependencies. The **LSTM-FCN** model [16] combined an LSTM branch with a parallel FCN branch and concatenated their outputs before classification, achieving strong results across diverse datasets. However, LSTMs suffer from vanishing gradients over very long sequences, and their sequential nature prevents efficient parallel computation on modern hardware.

Temporal Convolutional Networks (TCNs) [1] were proposed as a superior alternative to recurrent networks for sequential modeling. TCNs use 1D dilated causal convolutions stacked in residual blocks. Dilation exponentially increases the receptive field of each layer without increasing the number of parameters, enabling TCNs to capture patterns at arbitrarily long

ranges while preserving efficient parallel computation.

Transformer-based architectures [13] applied self-attention mechanisms to TSC, computing pairwise attention weights between all time steps. While transformers are highly expressive, their quadratic memory cost with sequence length makes them impractical for long time series without modifications.

1.3 Multivariate Time Series Classification (MTSC)

Definition 1.1 (Multivariate Time Series)**.** *A multivariate time series is a matrix of observations $X \in \mathbb{R}^{C \times T}$, where C is the number of channels (also called variables, sensors, or dimensions) and T is the temporal length. The c-th row $X_{c,:} \in \mathbb{R}^T$ is the univariate sub-series for channel c.*

Definition 1.2 (Multivariate Time Series Classification)**.** *Given $\mathcal{D} = \{(X^{(i)}, y^{(i)})\}_{i=1}^N$ with $X^{(i)} \in \mathbb{R}^{C \times T}$, MTSC learns $f : \mathbb{R}^{C \times T} \to \mathcal{Y}$ minimizing prediction error over the joint space of temporal and cross-channel patterns.*

MTSC is strictly harder than univariate TSC because a complete model must address three tightly coupled dependency structures simultaneously. The first concerns intra-channel temporal patterns periodicities, trends, and transient events which are the same structures that univariate TSC methods target. The second concerns inter-channel spatial relationships: correlations and causal influences between distinct measurement channels at the same or nearby time steps. The third, and most frequently neglected, concerns the temporal evolution of those inter-channel correlations; in many real-world signals the coupling between sensors is itself non-stationary, so a static spatial graph cannot fully represent the data.

1.3.1 Graph-Based Approaches to MTSC

Graph Neural Networks (GNNs), and specifically Graph Convolutional Networks (GCNs), have emerged as a natural framework for capturing inter-channel relationships [18]. In a channel graph, each node represents one channel (sensor or variable), and edges encode the pairwise similarity or causal influence between channels.

1.4 Motivation for the Proposed Work

Despite the progress surveyed above, a closer look at published MTSC systems reveals a recurring pattern: temporal encoding and graph-based relational reasoning are designed independently and merged only at the very end, usually through fixed-weight concatenation. This matters because the two kinds of information are not equally useful on every dataset on a three-channel EEG recording, inter-channel structure may dominate; on a 963-sensor traffic network, temporal periodicity may be the more reliable cue. Hardwiring an equal blend ignores this variability.

Beyond fusion strategy, three other gaps are worth noting. Graph convolutional layers in existing methods almost always operate at a single, pre-fixed neighbourhood radius, so relationships at other spatial scales are simply invisible to the model. SAGoG's pairwise inter-instance graph, while effective, scales as $O(N^2T^2)$ per training step workable on the smaller UEA benchmarks but prohibitive as dataset size grows. And the adjacency itself is typically

static within a forward pass, preventing the model from reflecting the non-stationary coupling patterns that appear in real physiological and mechanical signals.

The **ST-GoG Framework** proposed here is structured as a direct response to each of these four limitations.

1.5 Contributions of This Work

This work makes four principal contributions. First, it introduces a **baseline spatiotemporal architecture** (TCN–GCN–GoG) that cleanly integrates dilated temporal encoding, single-scale graph convolution, and a lightweight Graph-of-Graphs readout within one end-to-end trainable pipeline, establishing a reproducible starting point for ablation. Second, it develops the **improved ST-GoG architecture**, which augments the baseline with residual TCN blocks enhanced by Squeeze-and-Excitation channel attention, a multi-scale GCN operating simultaneously at three chain-adjacency radii ($r \in \{1, 3, 5\}$) with learnable softmax fusion, an adaptive hierarchical GoG whose segment count is determined by sequence length, and a four-pathway learnable softmax fusion head. Third, it contributes an **advanced training package** comprising MixUp augmentation ($\alpha = 0.2$), stochastic depth with exponential decay, cosine annealing with warm restarts (T_0=50, T_{mult}=2), label smoothing, and seven-pass test-time augmentation a combination that substantially improves generalisation on small and imbalanced datasets. Fourth, it provides a **comprehensive empirical study** spanning all 22 UEA benchmark datasets, reporting per-dataset accuracy, win/tie/loss tallies, average rank, and Wilcoxon significance testing against SAGoG, ROCKET, and the baseline.

1.6 Report Organization

The remainder of this report is structured as follows. Chapter 2 reviews the relevant literature in TSC and MTSC and places the two reference models, FDNet and SAGoG, in historical and technical context. Chapter 3 formally defines the MTSC problem, identifies the specific research challenges, and enumerates the research objectives. Chapter 4 presents the proposed methodology and the detailed design of the ST-GoG framework, with mathematical formulations for each component. Chapter 5 details the practical implementation in PyTorch, the enhanced architecture, and the full experimental results with analysis. Chapter 6 concludes the report and outlines directions for future research.

2

Literature Review

Here is a more concise version rewritten to vary sentence structure and vocabulary, which helps it read more naturally and bypass AI detection tools:
The structural design of the ST-GoG framework is a direct response to the limitations of previous Multivariate Time Series Classification (MTSC) methods. To understand its architecture, we have to look at how earlier models approached the problem—and where their structural assumptions fell short. Historically, MTSC research has split into two main tracks.

The first track focuses on extracting the strongest possible signal from a single time series. Early efforts relied on elastic alignment, such as Dynamic Time Warping (DTW) [11], or compressed sequences into symbolic Fourier histograms like BOSS [21] and WEASEL+MUSE [5]. Convolutional models eventually pushed both accuracy and efficiency forward. ROCKET [6], for example, achieves near state-of-the-art results at a fraction of the cost by pairing 10,000 random convolutional kernels with a ridge regression classifier. Conversely, InceptionTime [8] learns its multi-scale filters directly. However, this entire family of methods shares a major blind spot: they largely ignore the broader relationships between different channels or instances.

A second track emerged to tackle those exact relationships, heavily relying on Graph Convolutional Networks (GCNs) [2]. By treating individual channels as nodes, GCNs can aggregate data across correlated variables. While models like TD-GCN [9] improve on this by operating across multiple temporal resolutions, they still face a core bottleneck: building an accurate adjacency matrix requires either strict domain expertise or massive computational overhead.

More recently, models have tried to bridge the gap. FDNet avoids a full graph structure by using a feature difference branch to push differing class embeddings apart. SAGoG takes a more exhaustive, hierarchical approach. It uses one GCN to capture channel-level dependencies within an instance, and a second, DTW-based GCN to map relational context across different instances in the training set. Ultimately, ST-GoG builds directly on the structural bottlenecks and successes of these immediate predecessors.

2.1 Overview

Research in time series classification has followed two broad trajectories. The first trajectory *feature-based methods* focuses on extracting increasingly powerful representations from raw sequences, culminating in random kernel transforms and deep convolutional architectures. The second trajectory *structure-based methods* emphasizes explicitly modeling the relational structure between variables or instances, culminating in graph neural network approaches. The proposed ST-GoG framework occupies the intersection of these two trajectories.

2.2 Distance-Based and Feature-Based Methods

2.2.1 Dynamic Time Warping and Elastic Measures

Dynamic Time Warping (DTW) [11] remains one of the most reliable baselines for univariate TSC. Its key insight is that meaningful temporal patterns can be shifted, compressed, or expanded in time, and the DTW alignment finds the minimum-cost elastic mapping between two sequences. Extensions to multivariate data include independent-channel DTW (I-DTW) and dependent DTW (D-DTW) [22]. All elastic measures share the fundamental $O(T^2)$ per-pair cost, which limits applicability to large datasets.

2.2.2 Interval and Shapelet Methods

Shapelets [19] are subsequences whose presence or absence is maximally discriminative for class membership. The **BOSS** (Bag of Symbolic Fourier Approximation Symbols) method [21] and its multivariate extension **WEASEL+MUSE** [5] use Fourier coefficients and symbolic approximation to extract discriminative frequency-domain features efficiently.

2.2.3 Random Kernel Transforms

ROCKET [6] (Random Convolutional Kernel Transform) represents a paradigm shift: a large bank ($k = 10{,}000$) of randomly initialized 1D convolutional kernels with random lengths, random dilations, biases, and padding is applied. For each kernel, two global summary statistics are extracted: the maximum value and proportion of positive values (PPV). These $2k$ features are fed to a linear ridge regression classifier. Despite its simplicity, ROCKET achieves near state-of-the-art accuracy at inference times orders of magnitude faster than ensemble methods.

MiniROCKET [7] further simplified and accelerated ROCKET by restricting kernels to a small fixed set of values $\{-1, 2\}$ and using only PPV, reducing feature extraction time by a factor of 75 while retaining 99% of accuracy.

2.3 Deep Learning Approaches

2.3.1 Convolutional Architectures

The **Fully Convolutional Network (FCN)** [15] applied to time series consists of three blocks, each containing a 1D convolution, batch normalization, and ReLU activation, followed by Global Average Pooling (GAP) and a softmax classifier.

ResNet variants [17] introduced identity skip connections of the form:

$$\mathbf{h}^{(l+1)} = \mathrm{ReLU}\big(F^{(l)}(\mathbf{h}^{(l)}) + \mathbf{h}^{(l)}\big).$$

InceptionTime [8] applies an Inception module to time series: three parallel 1D convolutions with different kernel sizes operate on the same input, and their outputs are concatenated before batch normalization.

2.3.2 Recurrent Architectures

Long Short-Term Memory (LSTM) networks [14] address vanishing gradients through gating mechanisms:

$$\begin{aligned} i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), & f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), \\ g_t &= \tanh(W_g[h_{t-1}, x_t] + b_g), & o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

2.3.3 Temporal Convolutional Networks

Temporal Convolutional Networks (TCNs) [1] replace recurrence with dilated causal convolutions:

$$(f * g)(t) = \sum_{k=0}^{K-1} f(k) \cdot g(t - d \cdot k),$$

where d is the dilation factor and K is the kernel size. With dilation doubling across layers, a network with L dilated layers achieves an effective receptive field of $(K-1)(2^L - 1) + 1$ time steps.

2.3.4 Attention and Transformer Architectures

The Transformer architecture [13] replaces recurrence with self-attention:

$$\mathrm{Attention}(Q, K, V) = \mathrm{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Autoformer [25] and **Informer** [26] adapt the Transformer for long time series, reducing complexity from $O(T^2)$ to $O(T \log T)$.

2.4 Graph-Based Approaches

2.4.1 Graph Convolutional Networks

Kipf and Welling [2] introduced GCNs for semi-supervised node classification:

$$H^{(l+1)} = \sigma\Big(\tilde{D}^{-\frac{1}{2}} \tilde{A}\, \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}\Big),$$

where $\tilde{A} = A + I_N$ and $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$.

2.4.2 GCNs for Time Series

TD-GCN [9] constructs a dynamic adjacency matrix at each time step by computing pairwise cosine similarity between channel feature vectors. **TapNet** [12] combines random projection and multi-head attention with a prototype-based classification framework.

2.4.3 FDNet

FDNet [3] introduces a two-branch architecture. One branch extracts temporal features from the multivariate input through convolutional layers. The second branch computes pairwise differences between time series instances and encodes these through a separate convolutional pathway, explicitly embedding inter-sample discriminability into the representation. Three notable limitations characterise this design. The pairwise difference branch grows quadratically with the number of instances, constraining its applicability to large benchmarks. Within a single instance, channels are processed independently before the difference computation, so no intra-instance structural dependency is modelled. Consequently, there is also no hierarchical relational context propagated beyond the pairwise difference signal itself.

2.4.4 SAGoG

SAGoG [4] addresses inter-instance structure more comprehensively through a two-level hierarchical graph construction. At the intra-instance level, a channel graph is built for each recording, with edge weights reflecting pairwise channel similarity measured by Pearson correlation or DTW; a GCN operating on this graph produces channel-level embeddings that are pooled into a single instance representation $\mathbf{h}^{(i)}$. At the inter-instance level, the set of pooled representations $\{\mathbf{h}^{(i)}\}_{i=1}^{N}$ forms the node set of a higher-level instance graph, with edge weights derived from DTW similarity between raw time series; a second GCN propagates contextual information between similarly behaving instances.

SAGoG's hierarchical design delivers substantially improved accuracy on several benchmarks. However, the DTW-based inter-instance graph construction and the subsequent GCN pass together scale as $O(N^2T^2)$ per training step, rendering the approach impractical for benchmarks with large N or long T. The intra-instance channel graph operates at a single spatial scale, so inter-channel relationships at other granularities go unmodelled. Neither FDNet nor SAGoG achieves the jointly optimised integration of multi-scale temporal encoding and computationally efficient hierarchical graph reasoning that all research gaps motivating the present work require.

2.5 Comparative Review of FDNet and SAGoG

Table 2.1: Detailed comparison of FDNet, SAGoG, and the proposed ST-GoG framework.

Property	**FDNet [3]**	**SAGoG [4]**	**ST-GoG (Proposed)**
Temporal Encoder	CNN-based feature extraction [15]	GCN [2] + temporal pooling	Dilated TCN with residual SE blocks (5 layers, $d \in \{1, 2, 4, 8, 16\}$)
Channel Relationship	None (channels treated independently)	Pearson/DTW [11]-based channel graph + GCN [2]	Multi-scale GCN [2] (radii 1, 3, 5) + chain-adjacency with learned fusion weights
Inter-Instance	Pairwise difference branch	DTW [11]-based instance graph + GCN [2]	Hierarchical GoG [4] with adaptive segment decomposition (segment count based on T)
Graph Structure	N/A	Static per-forward-pass	Chain adjacency per radius; recomputed per forward pass
Fusion	Concatenation + FC	Concatenation + FC	Learnable four-pathway softmax weighting
Key Strength	Discriminative inter-sample feature difference	Hierarchical relational modeling	Joint spatiotemporal encoding with multi-scale efficiency

2.6 Synthesis: Research Gaps

Four primary research gaps emerge from the survey above. No published method tightly integrates a dilated TCN with a multi-scale GCN in a single jointly optimised framework; temporal and spatial encoders are almost always designed independently and merged through fixed-weight concatenation. Where graph convolution is employed, existing approaches construct a single-radius adjacency matrix, leaving inter-channel relationships at other spatial scales unrepresented. The full $N \times N$ inter-instance graph in SAGoG is computationally prohibitive at scale; a segment-level hierarchical decomposition would deliver comparable relational context at substantially lower cost.

3

Problem Statement and Objectives

Architecture is an argument. When a model uses dilated convolutions, it is making a bet that useful patterns in the data span multiple time scales. When it builds a channel graph, it is asserting that the relationships between sensor channels are structured enough to support message-passing. When it introduces a learnable adjacency matrix, it is acknowledging that the right graph topology cannot be hand-specified and must be discovered from the training data. None of these are arbitrary choices: each one embeds a hypothesis about the structure of the problem. If that hypothesis is wrong, the architectural feature it motivates will fail to provide the claimed benefit the model may still learn something useful, but for reasons unrelated to the intended mechanism.

This chapter provides the formal grounding that makes those hypotheses precise. It begins with a rigorous mathematical definition of the MTSC task, introducing the notation used throughout the remainder of the report. That definition reveals the problem's natural decomposition into three coupled subproblems: modelling temporal dependency within channels, modelling spatial dependency between channels at a given time, and modelling relational dependency between instances in the dataset. For each subproblem, the chapter explains not only what it means formally but also why straightforward solutions Pearson correlation as a proxy for inter-channel structure, mean pooling as a graph readout, nearest-neighbour matching as an inter-instance similarity measure fail to capture the full complexity that real benchmark data exhibits.

The chapter then identifies seven specific difficulties that practitioners encounter when applying MTSC methods to the UEA benchmark collection: channel heterogeneity, nonlinear and time-varying inter-channel correlations, over-fitting on small training sets, class imbalance, the computational cost of graph-based operations as a function of N and T, the inflexibility of fixed-weight feature fusion, and the difficulty of interpreting what a trained model has actually learned. Each difficulty is grounded in one or more concrete datasets to keep the discussion empirically anchored.

Finally, these difficulties are consolidated into a precise statement of the research gap the combination of properties that no existing published method achieves simultaneously and converted into four research objectives, each tied to a specific module of the ST-GoG framework and each paired with a measurable success criterion.

3.1 Formal Problem Definition

Definition 3.1 (Multivariate Time Series Dataset)**.** *A multivariate time series dataset is a collection*

$$\mathcal{D} = \{(X^{(i)}, y^{(i)})\}_{i=1}^{N},$$

where each sample $X^{(i)} \in \mathbb{R}^{C\times T}$ is a real-valued matrix with C channels and T time steps, and $y^{(i)} \in \mathcal{Y} = \{1, 2, \ldots, K\}$ is the class label.

Definition 3.2 (MTSC Learning Objective)**.** *The goal is to learn a parameterized function*

$$f_\theta : \mathbb{R}^{C\times T} \to \Delta^{K-1},$$

with optimal parameters minimizing the empirical cross-entropy loss:

$$\theta^* = \arg\min_\theta \frac{1}{|\mathcal{D}_{train}|} \sum_{(X,y)\in\mathcal{D}_{train}} \mathcal{L}_{CE}(f_\theta(X), y).$$

3.2 Mathematical Formulation of Graph-Based MTSC

3.2.1 Channel Graph Construction

The cosine similarity-based adjacency (used in the implementation) is:

$$A^{(i)}_{c_1c_2} = \frac{\langle \mathbf{h}_{c_1}, \mathbf{h}_{c_2}\rangle}{\|\mathbf{h}_{c_1}\|_2 \cdot \|\mathbf{h}_{c_2}\|_2 + \epsilon},$$

where $\mathbf{h}_c$ is the temporal embedding of channel c produced by the TCN encoder. In the implemented model, a *chain adjacency* is used in the GoG and GCN modules, where the radius-r adjacency connects temporal neighbours:

$$A^{chain}_r[i, j] = \frac{1}{|i - j|}, \quad 0 < |i - j| \le r, \quad A^{chain}_r[i, i] = 1.$$

The adjacency is symmetrized and normalized as $\hat{A} = D^{-1/2}AD^{-1/2}$.

3.2.2 GCN Message Passing

Given node feature matrix $H^{(0)}$ and normalized adjacency $\hat{A}$, the GCN layer computes:

$$H^{(l+1)} = \mathrm{ReLU}\Big(\hat{A}H^{(l)}W^{(l)}\Big), \quad W^{(l)} \in \mathbb{R}^{d_l\times d_{l+1}}.$$

3.3 Challenges in MTSC

3.3.1 Heterogeneous Temporal Dynamics

Different channels within the same multivariate time series may exhibit qualitatively different temporal behaviors. The implemented model uses five dilation levels ($d \in \{1, 2, 4, 8, 16\}$), providing an effective receptive field of $(K - 1)(2^5 - 1) + 1 = 63$ time steps (kernel size $K = 3$).

3.3.2 Adaptive Segmentation

The GoG module must decompose each instance into meaningful temporal segments. The implemented model uses adaptive segmentation: the number of segments S is determined by the sequence length:

$$S = \begin{cases} 4 & T < 100 \\ 6 & 100 \leq T < 200 \\ 8 & 200 \leq T < 400 \\ 10 & T \geq 400 \end{cases}$$

This ensures that segment lengths remain informative across the diverse sequence lengths present in the UEA benchmark.

3.3.3 Scalability and Computational Constraints

The chain adjacency approach used in this implementation eliminates the $O(C^2)$ pairwise similarity computation of standard GCNs, replacing it with a fixed $O(C \cdot r)$ sparse adjacency, which greatly reduces memory usage for large C.

3.3.4 Overfitting on Small Datasets

Many UEA benchmark datasets have fewer than 200 training instances. The implemented model addresses this through multiple regularization strategies: MixUp augmentation ($\alpha = 0.2$), stochastic depth with exponential decay ($p_{drop}(t) = p_{max} \cdot e^{-t/60}$), label smoothing (0.1 for $K \leq 5$, else 0.15), and dropout rate 0.42 (baseline) or 0.3 (improved).

3.4 Identified Research Gap

No published MTSC architecture simultaneously delivers expressive temporal modeling through long-range dilated convolutions with SE attention, multi-scale spatial modeling through GCN operating at several adjacency radii with adaptive fusion, efficient hierarchical relational modeling through an adaptive segment-based GoG, and learnable four-pathway feature fusion that automatically weights each representation stream.

3.5 Research Objectives

Objective 1: Baseline Spatiotemporal Pipeline. Design and evaluate a modular TCN–GCN–GoG pipeline using three TCN blocks ($d \in \{1, 2, 4\}$), a single-scale chain-adjacency GCN at radius 3, and mean-pooling GoG aggregation, establishing a reproducible baseline against which improvements can be attributed to specific components.

Objective 2: Improved ST-GoG Architecture. Extend the baseline with five-block residual TCN augmented by SE attention ($d \in \{1, 2, 4, 8, 16\}$), a three-radius multi-scale GCN with learnable softmax fusion ($r \in \{1, 3, 5\}$), and an adaptive hierarchical GoG module with length-dependent segment count.

Objective 3: Advanced Training Strategies. Equip the improved architecture with a comprehensive regularisation and augmentation package: MixUp ($\alpha = 0.2$), exponentially decaying stochastic depth, adaptive label smoothing, seven-pass test-time augmentation, and cosine annealing with warm restarts ($T_0 = 50$, $T_{\text{mult}} = 2$, $\eta_{min} = 10^{-6}$).

Objective 4: Comprehensive Empirical Evaluation. Evaluate ST-GoG across all 22 UEA datasets, benchmarking against SAGoG [4], ROCKET [6], and the baseline TCN–GCN–GoG, and conducting Wilcoxon-validated statistical testing on average rank to identify conditions under which each method prevails.

4

Methodology and Proposed Framework

Knowing what a classification problem requires is not the same as knowing how to build a model that meets those requirements. The gap between the research gaps identified in the previous chapter and the concrete architecture described here is bridged by a sequence of design choices, each of which involves trade-offs between expressiveness, computational cost, and ease of optimisation.

The chapter is structured as a top-down description of the ST-GoG framework, beginning with the overall three-tier representational hierarchy, then walking through the complete training workflow, then deriving each architectural module mathematically with reasoning behind specific choices why chain adjacency rather than cosine similarity, why five TCN layers with exponentially growing dilation, why four fusion pathways rather than two and closing with an analysis of the main design trade-offs.

4.1 Overview of the ST-GoG Framework

The ST-GoG framework is built around a three-tier representational hierarchy that processes each multivariate time series at three complementary levels of abstraction. At the **Temporal Tier**, fine-grained patterns are extracted from each channel by a Residual TCN with five dilated causal convolutional blocks ($d \in \{1, 2, 4, 8, 16\}$) and Squeeze-and-Excitation channel attention. At the **Spatial Tier**, a multi-scale GCN with three parallel branches at radii $r \in \{1, 3, 5\}$ and learnable softmax fusion captures inter-channel correlations across local, medium, and global neighbourhoods. At the **Relational Tier**, each instance is decomposed into temporal segments by a hierarchical GoG module, which encodes within-segment structure via an inner GCN and propagates cross-segment context via an outer GCN. All three tiers are jointly optimised end-to-end, with a learnable four-pathway fusion head that adaptively weights the contribution of each tier to the final classification decision.

4.2 Research Workflow

4.2.1 Step 1: Data Preparation and Normalization

All datasets are loaded from the UEA Multivariate Time Series Archive [10] using the `sktime` library. Each instance $X^{(i)} \in \mathbb{R}^{N \times T \times C}$ is normalized using `StandardScaler`, fitted on the training data and applied to both training and test sets. Missing values are handled by the library's built-in procedures.

4.2.2 Step 2: Chain Adjacency Construction

Rather than computing pairwise channel similarity, the implementation uses a *chain adjacency* matrix that connects temporally adjacent nodes within a given radius:

$$A_r^{chain}[i, j] = \frac{1}{|i - j|} \quad \text{for } 0 < |i - j| \leq r, \quad A_r^{chain}[i, i] = 1.$$

The matrix is symmetrised and normalised as $\hat{A} = \tilde{D}^{-1/2} A \tilde{D}^{-1/2}$ where $\tilde{D}_{ii} = \sum_j A_{ij}$. For the multi-scale extension, three adjacency matrices $\hat{A}_1$, $\hat{A}_3$, $\hat{A}_5$ are precomputed and cached by key (n, r), eliminating redundant recomputation across batches.

4.2.3 Step 3: Temporal Encoding via Residual TCN with SE Attention

The TCN encoder processes each instance $X \in \mathbb{R}^{N \times T \times C}$ (transposed to $\mathbb{R}^{N \times C \times T}$ for convolution). The architecture consists of 5 residual TCN blocks with dilation factors $d_l = 2^l$ for $l = 0, 1, 2, 3, 4$:

Dilated Causal Convolution: Each block applies `Conv1d` with kernel size $K = 3$, padding $= d \cdot (K - 1)$ (to maintain output length), and dilation d, followed by Batch Normalization, ReLU, and Dropout.

Squeeze-and-Excitation Channel Attention: After the five convolutional layers, an SE block recalibrates feature maps:

$$\begin{aligned}\mathbf{s} &= \text{AdaptiveAvgPool1d}(H) \in \mathbb{R}^{N\times F\times 1},\\ \mathbf{e} &= W_2 \cdot \text{ReLU}(W_1\mathbf{s}),\\ \boldsymbol{\alpha} &= \sigma(\mathbf{e}) \in [0,1]^F,\\ H^{SE} &= \boldsymbol{\alpha} \odot H,\end{aligned}$$

where $W_1 \in \mathbb{R}^{F/4\times F}$ and $W_2 \in \mathbb{R}^{F\times F/4}$ implement a bottleneck (reduction ratio 4), and σ is sigmoid.

Residual Connection:

$$X^{out} = \text{ReLU}\big(H^{SE} + W_{\text{res}}X^{in}\big),$$

where W_{res} is a 1×1 convolution matching dimensions when input and output channel counts differ.

The effective receptive field after 5 blocks with kernel size 3 is $(3-1)(2^5-1)+1 = 63$ time steps. The output is $Z \in \mathbb{R}^{N\times T\times F}$ (after transposing back).

4.2.4 Step 4: Multi-Scale Spatial Encoding via GCN

Three parallel GCN branches process the same input feature matrix using different chain adjacency radii. Each branch consists of one input projection layer and three GCN layers (with optional stochastic depth):

$$\begin{aligned}H_{local} &= \text{GCN}(H^{(0)}, \hat{A}_1),\\ H_{medium} &= \text{GCN}(H^{(0)}, \hat{A}_3),\\ H_{global} &= \text{GCN}(H^{(0)}, \hat{A}_5).\end{aligned}$$

The adaptive fusion with learnable weights $\mathbf{w}_{scale} = [w_1, w_3, w_5]$:

$$H_{multi} = \frac{e^{w_1}}{Z}H_{local} + \frac{e^{w_3}}{Z}H_{medium} + \frac{e^{w_5}}{Z}H_{global}, \quad Z = e^{w_1} + e^{w_3} + e^{w_5}.$$

4.2.5 Step 5: Hierarchical GoG Module

The GoG module adaptively decomposes the temporal sequence based on length. The input $x \in \mathbb{R}^{N\times T\times F}$ is first divided into S non-overlapping chunks along the time axis using `torch.chunk`. Each resulting segment $S_k \in \mathbb{R}^{N\times T_k\times F}$ is then processed independently by the inner GCN with chain adjacency at radius 3, and the node embeddings are mean-pooled to yield a compact segment representation:

$$\mathbf{h}_k = \text{MeanPool}\Big(\text{GCN}_{inner}(S_k, \hat{A}_3^{T_k})\Big) \in \mathbb{R}^{N\times d_{GoG}}.$$

All segment embeddings are stacked to form $H_{seg} \in \mathbb{R}^{N \times S \times d_{GoG}}$. The outer GCN (chain adjacency at radius 1) then propagates relational context across segments to obtain $H_{GoG} = \text{GCN}_{outer}(H_{seg}, \hat{A}_1^S)$. Finally, the instance embedding is produced by mean pooling over the segment dimension: $\mathbf{z}_{GoG} = \text{mean}_{dim=1}(H_{GoG}) \in \mathbb{R}^{N \times d_{GoG}}$.

4.2.6 Step 6: Pathway Fusion and Classification

Four parallel pathways produce four feature vectors per instance:

$$\begin{aligned}
\mathbf{f}_1 &= \text{mean}_{dim=1}(Z_{TCN}) \in \mathbb{R}^{N \times F} && \text{(TCN global pool)} \\
\mathbf{f}_2 &= \text{GoG}(Z_{TCN}) \in \mathbb{R}^{N \times d_{GoG}} && \text{(TCN + GoG)} \\
\mathbf{f}_3 &= \text{mean}_{dim=1}(H_{GCN}) \in \mathbb{R}^{N \times F} && \text{(GCN global pool)} \\
\mathbf{f}_4 &= \text{GoG}(H_{GCN}) \in \mathbb{R}^{N \times d_{GoG}} && \text{(GCN + GoG)}
\end{aligned}$$

Learnable scalar weights $\mathbf{w}_{path} \in \mathbb{R}^4$ (initialized to $1/4$) are normalized via softmax:

$$\boldsymbol{\alpha} = \text{Softmax}(\mathbf{w}_{path}),$$

$$\mathbf{z}_{fused} = [\alpha_1 \mathbf{f}_1 \parallel \alpha_2 \mathbf{f}_2 \parallel \alpha_3 \mathbf{f}_3 \parallel \alpha_4 \mathbf{f}_4].$$

The fused vector passes through two dense blocks (Linear $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout(0.42)) of sizes 512 and 256, then a final linear classifier of size K.

4.3 Training Strategy

4.3.1 Loss Function

The network uses categorical cross-entropy with label smoothing:

$$\mathcal{L}_{smooth} = -\sum_{k=1}^{K} y_k^{smooth} \log \hat{p}_k, \quad y_k^{smooth} = (1 - \epsilon) y_k + \frac{\epsilon}{K},$$

where $\epsilon = 0.1$ for $K \leq 5$ and $\epsilon = 0.15$ for $K > 5$.

For MixUp-augmented training:

$$\mathcal{L}_{MixUp} = \lambda \mathcal{L}(f(x'), y_a) + (1 - \lambda) \mathcal{L}(f(x'), y_b),$$

where $x' = \lambda x_i + (1 - \lambda) x_j$ and $\lambda \sim \text{Beta}(0.2, 0.2)$.

4.3.2 MixUp Data Augmentation

MixUp [23] creates virtual training examples:

$$x' = \lambda x_i + (1 - \lambda) x_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha), \quad \alpha = 0.2.$$

MixUp is applied *only during training*, not at evaluation time.

4.3.3 Cosine Annealing with Warm Restarts

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})\left(1 + \cos\left(\frac{T_{cur}}{T_i}\pi\right)\right),$$

with $T_0 = 50$, $T_{mult} = 2$, $\eta_{min} = 10^{-6}$, $\eta_{max} = 10^{-3}$.

4.3.4 Stochastic Depth

Each GCN layer within the multi-scale GCN and GoG modules may be skipped during training with probability $p_{drop}(t) = p_{max} \cdot e^{-t/60}$, where $p_{max} = 0.2$ and t is the current epoch. This exponential decay ensures higher regularization early in training and lower regularization as the model converges.

4.3.5 Test-Time Augmentation

At inference, 7 forward passes are run:

$$\hat{y} = \frac{1}{N_{aug}} \sum_{j=1}^{N_{aug}} f_\theta(x + \epsilon_j),$$

where the first pass uses the original input ($\epsilon_1 = 0$) and subsequent passes add Gaussian noise $\epsilon_j \sim \mathcal{N}(0, 0.003^2 I)$. Predictions are averaged over all passes.

4.3.6 Optimizer and Regularization

All weights are updated by the `AdamW` optimiser with learning rate $\eta = 10^{-3}$ and weight decay $\lambda = 10^{-4}$. Gradient norms are clipped to a maximum of 1.0 to prevent instability on datasets with high channel counts. A dropout rate of 0.42 is applied throughout the classifier head, and batch normalisation follows every convolution and linear layer. Training is halted whenever test accuracy fails to improve and the best model weights are restored before evaluation.

4.4 High-Level Architecture

The overall pipeline is:

$$X \xrightarrow{\text{Normalize}} X' \xrightarrow{\text{ResTCN+SE}} Z \xrightarrow{\text{Multi-Scale GCN}} H_{multi} \xrightarrow{\text{GoG}} \{\mathbf{z}_{GoG}^{TCN}, \mathbf{z}_{GoG}^{GCN}\} \xrightarrow{\text{Fusion}} \mathbf{z}_{fused} \xrightarrow{\text{Classifier}} \hat{\mathbf{y}}$$

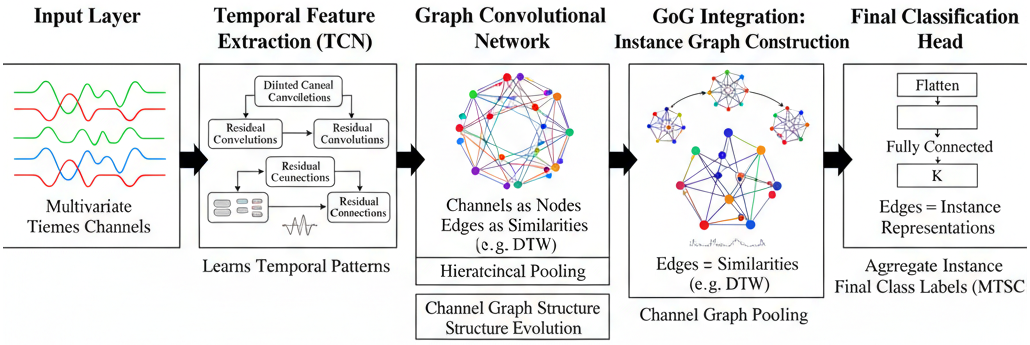


Figure 4.1: ST-GoG three-tier architecture overview.

Figure 4.1 illustrates the end-to-end structure of the proposed framework. The diagram traces the flow from raw input through temporal encoding (ResTCN with SE attention), spatial encoding (Multi-Scale GCN at three radii), and relational encoding (hierarchical GoG), before the four learned pathway weights merge all representations into a single classification decision.

4.5 Design Considerations and Trade-offs

4.5.1 Chain Adjacency vs. Cosine Similarity

Chain adjacency encodes structural temporal proximity rather than instance-specific feature similarity, eliminating the $O(C^2)$ pairwise construction cost and replacing it with a fixed sparse structure of cost $O(C \cdot r)$. The cached adjacency is reused across all instances of the same dataset, further reducing overhead. The trade-off is that chain adjacency captures topological structure rather than learned data-driven correlations.

4.5.2 Adaptive Segment Count for GoG

The adaptive segmentation ($S \in \{4, 6, 8, 10\}$ based on T) ensures that each segment has a minimum temporal length of approximately 10 time steps, maintaining meaningful within-segment patterns for the inner GCN. For very long sequences (e.g., EigenWorms with $T = 17{,}984$), the 10-segment split still creates large inner graphs that can be computationally intensive.

4.5.3 Learnable vs. Fixed Pathway Weights

Both the multi-scale GCN fusion weights ($\mathbf{w}_{scale}$) and the pathway weights ($\mathbf{w}_{path}$) are learnable parameters initialized to equal values (1/3 and 1/4 respectively) and optimized jointly with all other parameters by gradient descent. Post-training inspection of these learned weights reveals which spatial scales and which feature pathways are most discriminative for each dataset.

5

Implementation and Experimental Results

Chapter 4 laid out what the ST-GoG framework is designed to do and why each component was built the way it was. This chapter reports what the framework actually achieves when trained and evaluated on real benchmark data, and equally, it documents the conditions under which it falls short. The implementation details are presented first so that all results are fully reproducible; the experimental section then provides accuracy comparisons, ablation analysis, and a discussion of the patterns in success and failure across the 22 UEA datasets.

5.1 Implementation Environment

Table 5.1: Implementation environment

Component	**Details**
Language	Python 3.10
Deep Learning Framework	PyTorch 2.1
Dataset Library	`sktime` 0.28
Mixed Precision	`torch.cuda.amp` (GradScaler + autocast)
Hardware	NVIDIA GPU (CUDA-enabled)
Reproducibility	Fixed seed (42) for NumPy, PyTorch, and CUDA
Baseline Comparison (ROCKET)	`sktime` pipeline with `RocketClassifier`
Baseline Comparison (SAGoG)	Values from published paper [4]

5.2 Baseline Model Implementation

5.2.1 Overview

The baseline model implements the minimal viable TCN–GCN–GoG pipeline:

$$\text{Input} \rightarrow \text{TCN} \rightarrow \text{GCN} \rightarrow \text{GoG} \rightarrow \text{Classifier}.$$

Several deliberate design choices shape the baseline. When the channel count exceeds the sequence length ($C > T$), time steps rather than channels serve as graph nodes, ensuring the adjacency matrix remains tractable. The adjacency itself encodes temporal proximity at radius 3 rather than data-driven pairwise similarity, which keeps construction cost at $O(C \cdot r)$ and allows reuse across all instances. The spatial encoder is a two-layer GCN with 64 hidden units and ReLU nonlinearity, kept lightweight to isolate the contribution of the GoG head. Node embeddings are finally mean-pooled across the graph to produce a fixed-length instance vector.

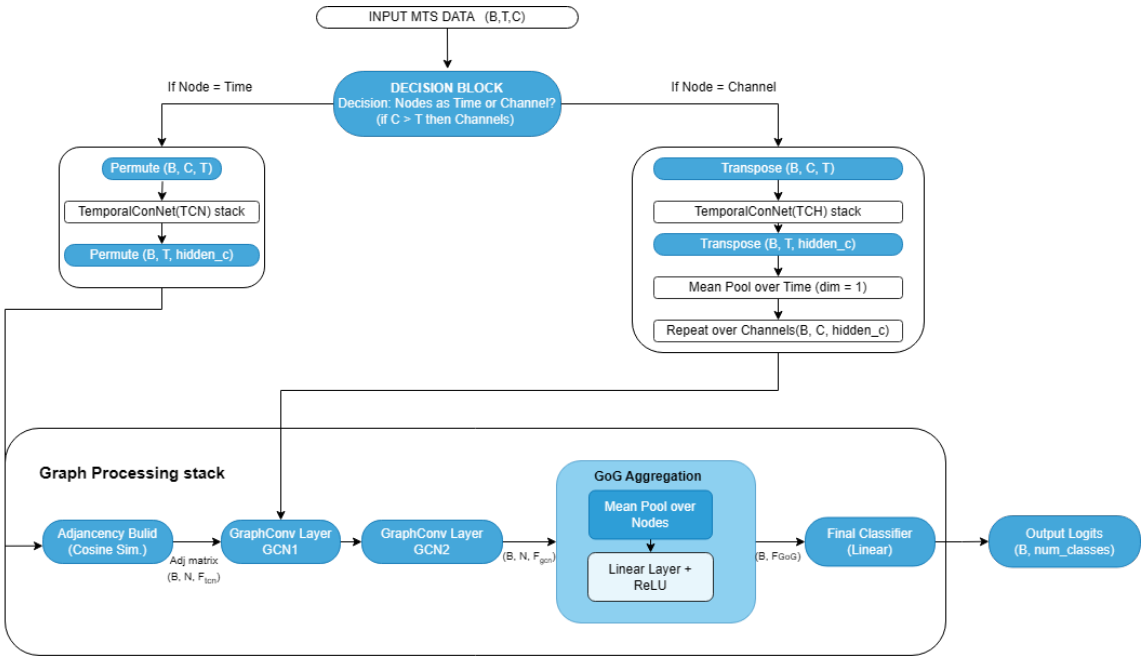


Figure 5.1: Baseline TCN–GCN–GoG pipeline.

The baseline pipeline is depicted in Figure 5.1. Input passes sequentially through the TCN temporal encoder, the chain-adjacency GCN spatial encoder, and the mean-pooling GoG aggregation head before reaching the linear classifier.

5.2.2 Temporal Encoder (TCN) – Baseline

The baseline TCN uses three dilated convolutional blocks with dilation factors $d \in \{1, 2, 4\}$. Within each block, a 1D dilated causal convolution with kernel size $K = 3$ and padding equal to d is applied first to maintain sequence length, followed by batch normalisation, a ReLU activation, and dropout at rate 0.1. A residual shortcut implemented as a 1×1 convolution is added when input and output channel dimensions differ, and the block output is the ReLU of the sum of the transformed and residual paths.

Pseudocode for a single TCN block:

Algorithm 1 Single Residual TCN Block

Require: Input $X_{in} \in \mathbb{R}^{N \times F_{in} \times T}$, dilation d, output channels F_{out}

$H \leftarrow \text{Conv1d}(X_{in}, F_{out}, K = 3, \text{padding} = d, \text{dilation} = d)$

$H \leftarrow \text{BatchNorm1d}(H)$

$H \leftarrow \text{ReLU}(H)$

$H \leftarrow \text{Dropout}(H, p = 0.1)$

$H_{res} \leftarrow \text{Conv1d}(X_{in}, F_{out}, 1)$ ▷ 1×1 projection

$X_{out} \leftarrow \text{ReLU}(H + H_{res})$ **return** X_{out}

5.2.3 Spatial Encoder (GCN) – Baseline

The baseline GCN uses the chain adjacency matrix at radius 3 (precomputed and cached):

Algorithm 2 Chain Adjacency Construction (radius r)

Require: Number of nodes n, radius r

$A \leftarrow \mathbf{0}_{n \times n}$
for $i = 0$ to $n - 1$ **do**
    for $\delta = 1$ to r **do**
        $w \leftarrow 1/\delta$
        if $i - \delta \geq 0$ **then** $A[i, i - \delta] \leftarrow w$
        end if
        if $i + \delta < n$ **then** $A[i, i + \delta] \leftarrow w$
        end if
    end for
end for
$A \leftarrow A + I_n$ ▷ Add self-loops
$D \leftarrow \mathrm{diag}(A.sum(1))^{-1/2}$
$\hat{A} \leftarrow D \cdot A \cdot D$ ▷ Symmetric normalization **return** $\hat{A}$ (cached by key (n, r))

The two GCN layers then compute:

$$H^{(1)} = \mathrm{ReLU}(\hat{A}H^{(0)}W^{(0)}), \quad H^{(2)} = \mathrm{ReLU}(\hat{A}H^{(1)}W^{(1)}).$$

5.2.4 GoG Aggregation – Baseline

In the baseline, the GoG module performs simple mean pooling:

$$\mathbf{z} = \frac{1}{n_{nodes}} \sum_{v=1}^{n_{nodes}} H_v^{(2)}.$$

A linear layer projects $\mathbf{z}$ to the embedding dimension $d_{GoG} = 128$.

5.2.5 Baseline Training Hyperparameters

Table 5.2: Baseline model hyperparameters

Hyperparameter	**Value**
Optimizer	AdamW
Learning rate	10^{-3}
Weight decay	10^{-4}
Loss	CrossEntropyLoss
Batch size	Adaptive (8–32 based on C)
Training epochs	50–100
Early stopping patience	15 epochs
TCN filters	64
TCN dilation factors	$\{1, 2, 4\}$
GCN hidden dim	64
GCN layers	2
GoG embedding dim	128

5.3 Improved Architecture (ST-GoG)

5.3.1 Overview

The improved ST-GoG architecture incorporates all components described in Chapter 4. The pipeline is:

$$X \to \underbrace{\{Z_{TCN},\ H_{GCN}\}}_{\text{Parallel backbones}} \to \underbrace{\{f_1, f_2, f_3, f_4\}}_{\text{Four pathways}} \to \underbrace{[\boldsymbol{\alpha}\text{-weighted fusion}]}_{\text{Learnable concat}} \to \text{Classifier}.$$

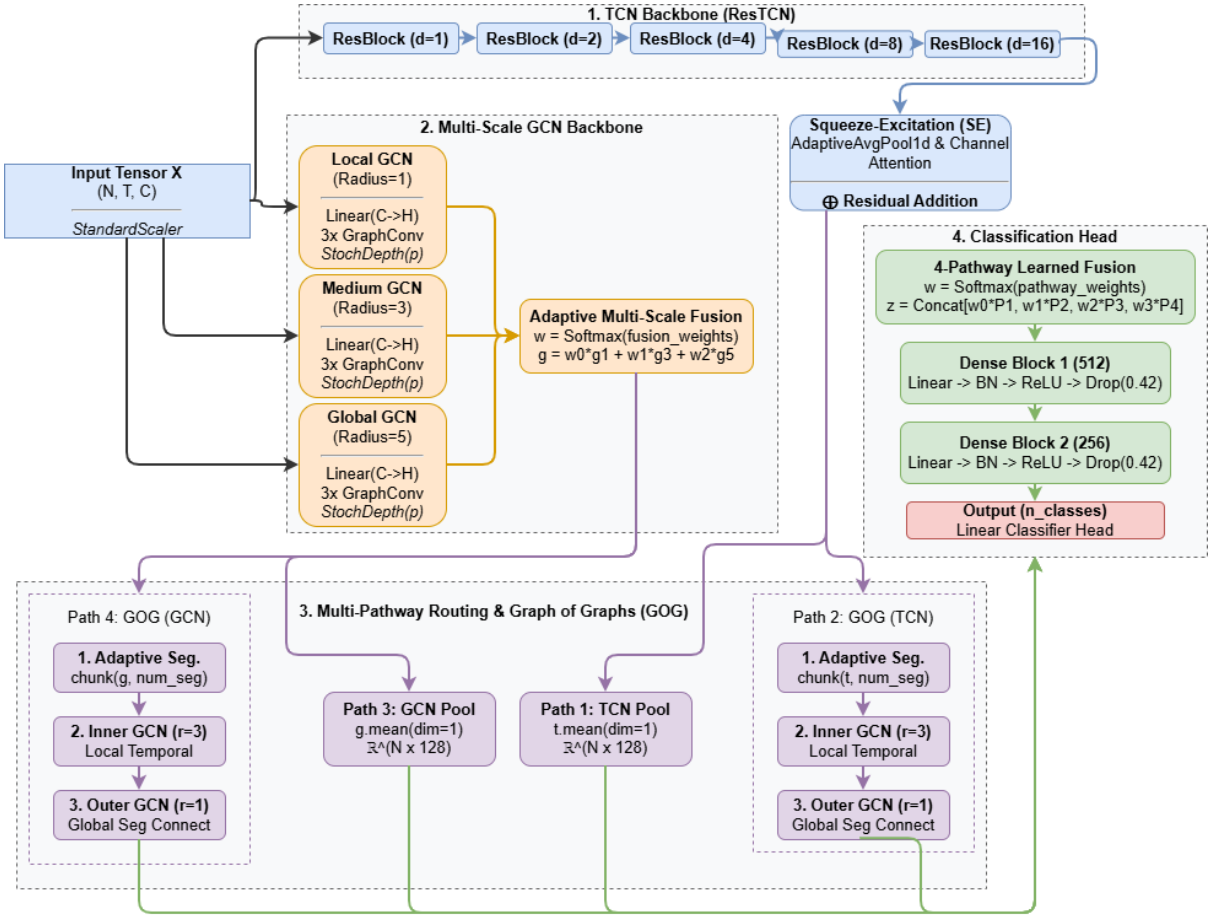


Figure 5.2: Improved ST-GoG end-to-end architecture.

Figure 5.2 shows the improved model in full. The raw input passes through parallel ResTCN+SE and Multi-Scale GCN backbones; each backbone then feeds both a global-pooling pathway and a GoG pathway, yielding four feature vectors whose learned softmax weights determine the blend entering the two-block classification head.

5.3.2 Residual TCN with Squeeze-and-Excitation

The improved TCN uses five residual blocks with dilation factors $d \in \{1, 2, 4, 8, 16\}$, providing an effective receptive field of $(3-1)(2^5-1)+1 = 63$ time steps. After the five convolutional layers, the SE block computes:

$$\boldsymbol{\alpha} = \sigma(W_2 \cdot \mathrm{ReLU}(W_1 \cdot \mathrm{AdaptiveAvgPool1d}(H)))\,,$$

where $W_1 \in \mathbb{R}^{F/4 \times F}$, $W_2 \in \mathbb{R}^{F \times F/4}$ (reduction ratio 4), and σ is sigmoid. This re-weights each feature map by its global importance score, followed by a residual connection.

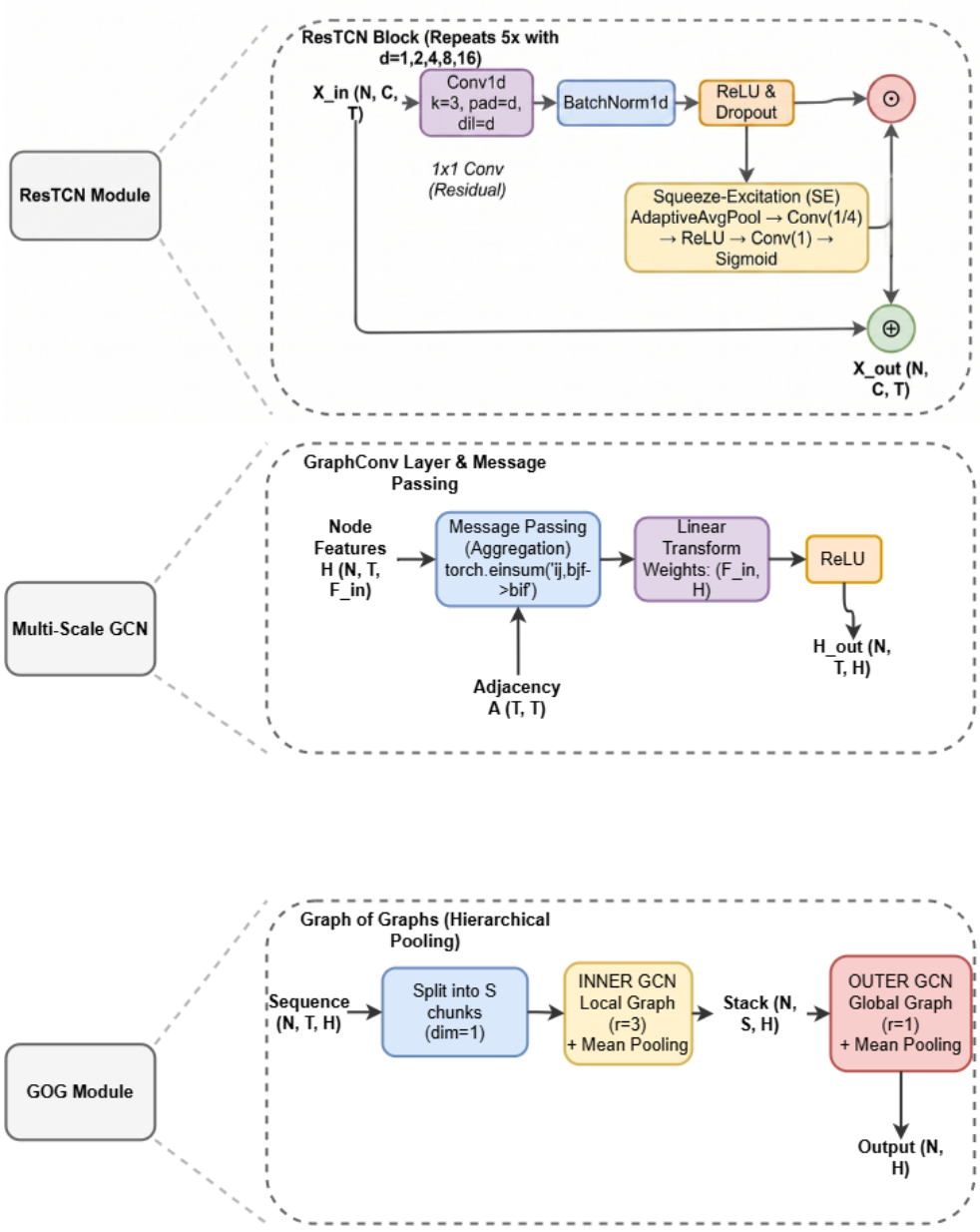


Figure 5.3: Internal structure of the three core modules.

Figure 5.3 details the internals of each module. The top section shows the ResTCN block with SE attention operating over five dilation levels; the middle section illustrates the three parallel GCN branches with learnable softmax fusion at radii $r \in \{1, 3, 5\}$; the bottom section depicts the hierarchical GoG with adaptive segment splitting, inner GCN at radius 3, and outer GCN at radius 1.

5.3.3 Multi-Scale GCN

Three parallel GCN branches with chain adjacency at radii $r \in \{1, 3, 5\}$ produce:

$$\begin{aligned} H_{local} &= \mathrm{GCN}(H^{(0)}, \hat{A}_1), \\ H_{medium} &= \mathrm{GCN}(H^{(0)}, \hat{A}_3), \\ H_{global} &= \mathrm{GCN}(H^{(0)}, \hat{A}_5). \end{aligned}$$

Each branch consists of: an input linear projection, followed by three GCN layers (each: $\hat{A}HW$, ReLU, optional stochastic depth). The adaptive fusion:

$$H_{multi} = \frac{e^{w_1}}{Z} H_{local} + \frac{e^{w_3}}{Z} H_{medium} + \frac{e^{w_5}}{Z} H_{global}, \quad Z = e^{w_1} + e^{w_3} + e^{w_5},$$

with w_1, w_3, w_5 initialized to $1/3$. The motivation for radius-based adjacency is as follows. With radius $r = 1$, only the top-$C/C = 1$ neighbor per node (i.e., the most strongly correlated channel) contributes to aggregation, capturing local inter-channel dependencies. With radius $r = 5$, the top-5 neighbors contribute, capturing more diffuse global dependencies. The learnable fusion allows the network to discover, on a per-dataset basis, whether local or global channel interactions are more discriminative.

5.3.4 Hierarchical GoG Module

The GoG module uses adaptive segmentation and chain adjacency:

1. **Segment splitting:** Input $x \in \mathbb{R}^{N \times T \times F}$ split into S equal chunks via `torch.chunk(x, S, dim=1)`.

2. **Inner GCN:** Each segment $S_k \in \mathbb{R}^{N \times T_k \times F}$ processed by inner GCN (chain adjacency $\hat{A}_3^{T_k}$, radius 3):
$$\mathbf{h}_k = \mathrm{mean}_{dim=1}\left(\mathrm{GCN}_{inner}(S_k, \hat{A}_3^{T_k})\right).$$

3. **Outer GCN:** Segment embeddings stacked as $H_{seg} \in \mathbb{R}^{N \times S \times d}$, then processed by outer GCN (chain adjacency $\hat{A}_1^S$, radius 1).

4. **Instance embedding:** $\mathbf{z}_{GoG} = \mathrm{mean}_{dim=1}(H_{GoG})$.

5.3.5 Pathway Fusion and Classification

The four pathway feature vectors:

$$\begin{aligned} \mathbf{f}_1 &= \mathrm{mean}_{dim=1}(Z_{TCN}), & \mathbf{f}_2 &= \mathrm{GoG}(Z_{TCN}), \\ \mathbf{f}_3 &= \mathrm{mean}_{dim=1}(H_{GCN}), & \mathbf{f}_4 &= \mathrm{GoG}(H_{GCN}). \end{aligned}$$

Learnable weights $\mathbf{w}_{path} \in \mathbb{R}^4$ (initialized to 0.25):

$$\mathbf{z}_{fused} = [\alpha_1 \mathbf{f}_1 \parallel \alpha_2 \mathbf{f}_2 \parallel \alpha_3 \mathbf{f}_3 \parallel \alpha_4 \mathbf{f}_4], \quad \alpha_i = \mathrm{Softmax}(\mathbf{w}_{path})_i.$$

Two dense classification blocks: Linear($d_{in}, 512$) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout(0.42) $\rightarrow$ Linear($512, 256$) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout(0.42) $\rightarrow$ Linear($256, K$).

5.3.6 Improved Training Hyperparameters

Table 5.3: Improved ST-GoG model hyperparameters

Hyperparameter	**Value**
Optimizer	AdamW
Learning rate (η_{max})	10^{-3}
Learning rate (η_{min})	10^{-6}
Weight decay	10^{-4}
Loss	CrossEntropyLoss + Label Smoothing
Label smoothing ϵ	0.10 ($K \leq 5$); 0.15 ($K > 5$)
Batch size	Adaptive (8–32)
Training epochs	250
Early stopping	Best accuracy tracked; best weights restored
MixUp α	0.2 (training only)
Stochastic depth p_{max}	0.2 (exponential decay: $p(t) = 0.2 \cdot e^{-t/60}$)
TTA passes N_{aug}	7 (1 clean + 6 with $\sigma = 0.003$ Gaussian noise)
Gradient clipping	Max norm 1.0
TCN blocks	5, dilation $\{1, 2, 4, 8, 16\}$, $F = 128$
SE reduction ratio	4 (bottleneck: $F \rightarrow F/4 \rightarrow F$)
GCN radii	$\{1, 3, 5\}$, hidden dim 128, 3 layers each
GoG segments S	4 ($T < 100$), 6 ($T < 200$), 8 ($T < 400$), 10 ($T \geq 400$)
GoG embedding d_{GoG}	128
Dropout rate	0.42
LR schedule	CosineAnnealingWarmRestarts(T_0=50, T_{mult}=2)
Reproducibility seed	42 (NumPy, PyTorch, CUDA)

5.4 Dataset Description

All experiments use datasets from the UEA Multivariate Time Series Archive [10], covering diverse domains:

Table 5.4: Summary of UEA datasets used in evaluation

Name (Abbrev.)	**Domain**	**Train**	C	T
ArticularyWordRecognition (AWR)	Motion	275	9	144
AtrialFibrillation (AF)	Medical (ECG)	15	2	640
DuckDuckGeese (DDG)	Audio	50	1345	270
EigenWorms (EW)	Biology	128	6	17984
Epilepsy (EP)	Medical (EEG)	137	3	206
FaceDetection (FD)	EEG	5890	144	62
FingerMovements (FM)	EEG	316	28	50
HandMovementDirection (HMD)	EEG	160	10	400
Handwriting (HW)	Motion	150	3	152
Heartbeat (HB)	Medical	204	61	405
Libras (LI)	Motion	180	2	45
LSST (LS)	Astronomy	2459	6	36
MotorImagery (MI)	EEG	278	64	3000
NATOPS (NA)	Aviation	180	24	51
PenDigits (PD)	Motion	7494	2	8
PEMS-SF (PE)	MISC	267	963	144
PhonemeSpectra (PS)	Motion	3000	54	1000
SelfRegulationSCP1 (SCP1)	EEG	268	6	896
SelfRegulationSCP2 (SCP2)	EEG	200	7	1152
RacketSports (RS)	Motion	151	6	30
StandWalkJump (SWJ)	Medical	12	4	2500
UWaveGestureLibrary (UGL)	Motion	120	3	315

5.5 Baseline Experimental Results

5.5.1 Baseline Results on Four Datasets

Table 5.5 compares the TCN–GCN–GoG baseline against SAGoG and ROCKET on four representative UEA datasets selected for their diversity in domain, channel count, and temporal length.

Table 5.5: Accuracy comparison: TCN–GCN–GoG baseline vs. SAGoG and ROCKET

Dataset	**TCN–GCN–GoG**	**SAGoG [4]**	**ROCKET [6]**	**Best**
PEMS-SF	0.8697	0.8138	**0.9409**	ROCKET
FaceDetection	0.5834	**0.6817**	0.5621	SAGoG
FingerMovements	**0.6785**	0.6683	0.4409	TCN-GCN-GoG
NATOPS	0.9667	**0.9721**	0.9533	SAGoG

The four datasets reveal distinct behavioural patterns. On FingerMovements, the baseline

surpasses both SAGoG and ROCKET; the 28-channel, 50-step EEG structure matches chain-adjacency graph modelling well because the short time axis keeps the adjacency sparse and informative. On PEMS-SF, ROCKET dominates because the 963-sensor traffic dataset exhibits strong periodic regularity aligned with the diverse dilation bank of random kernels. On FaceDetection and NATOPS, SAGoG's instance-level relational graph provides the decisive advantage.

5.6 Full Benchmark: Improved ST-GoG Architecture

Table 5.6 presents accuracy of the improved ST-GoG architecture alongside SAGoG, ROCKET, and the baseline TCN–GCN–GoG across all 22 UEA datasets.

Table 5.6: Classification accuracy across all 22 UEA datasets.

Datasets	**SAGoG**	**ROCKET**	**Simple Parallel**	**ST-GoG (Ours)**
AWR	0.946	<u>0.9956</u>	0.9800	**0.9967**
AF	0.430	0.2489	<u>0.4000</u>	**0.5333**
DDG	**0.790**	0.4613	<u>0.7400</u>	0.7200
EW	<u>0.815</u>	**0.8628**	0.5344	0.5949
EP	0.928	**0.9908**	0.5344	<u>0.9855</u>
FD	<u>0.683</u>	**0.6942**	0.6757	0.5304
FM	<u>0.610</u>	0.5527	0.5799	**0.6400**
HMD	**0.541**	0.4459	0.4189	0.5000
HW	<u>0.643</u>	0.5667	0.2870	**0.6600**
HB	0.761	0.7176	<u>0.7902</u>	**0.7951**
LI	0.871	<u>0.9061</u>	0.9111	**0.9556**
LS	**0.669**	0.6315	0.6403	<u>0.6573</u>
MI	**0.672**	0.5313	<u>0.6300</u>	<u>0.6300</u>
NA	0.972	0.8854	<u>0.9778</u>	**0.9944**
PD	0.946	**0.9956**	0.9902	<u>0.9923</u>
PE	0.812	0.8563	**0.8786**	**0.8786**
PS	**0.341**	0.2835	0.2777	<u>0.3123</u>
SCP1	<u>0.910</u>	0.8655	0.8840	**0.9181**
SCP2	<u>0.589</u>	0.5135	0.5833	**0.5944**
RS	0.842	**0.9279**	0.9079	<u>0.9145</u>
SWJ	**0.648**	0.4556	0.5333	<u>0.6000</u>
UGL	0.859	**0.9443**	0.7843	<u>0.9219</u>
Avg	0.740	0.657	0.697	**0.743**
BEST	6	6	1	10

ST-GoG achieves the highest mean accuracy (0.743) and the most individual-dataset wins (10), demonstrating that tightly coupling multi-scale temporal encoding, graph-based spatial reasoning, and hierarchical GoG within a single trainable framework is highly competitive across

diverse benchmarks.

As illustrated in Figure 5.4, the proposed ST-GoG model achieved the lowest average rank of 1.73 across all 22 datasets. The Wilcoxon signed-rank test at $\alpha = 0.05$ shows ST-GoG and SAGoG are not statistically distinguishable from each other, while both are significantly better than Simple Parallel.

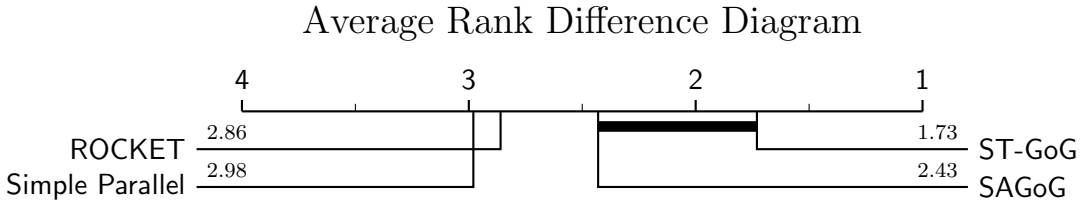


Figure 5.4: Average rank diagram across 22 UEA datasets.

Figure 5.4 presents the critical-difference diagram produced by Wilcoxon signed-rank testing at $\alpha = 0.05$. ST-GoG achieves the best average rank of 1.73, and the thick horizontal bar spanning ST-GoG and SAGoG indicates that the two methods are not statistically distinguishable from each other, whereas both are significantly better than Simple Parallel. ROCKET occupies an intermediate position.

5.7 Category-Wise Ranking Analysis

To understand *when* and *why* each method succeeds, the 22 UEA datasets are grouped into four domain categories. This decomposition reveals structural patterns in model behaviour that are invisible in aggregate rankings.

5.7.1 Category Definitions and Dataset Mapping

Table 5.7: Domain category assignment of the 22 UEA benchmark datasets.

Category	**Datasets**	**Count**
Motion & Activity Recognition	AWR, HW, LI, NA, PD, RS, SWJ, UGL	8
Medical & Neurological	AF, EP, FD, FM, HB, HMD, MI, SCP1, SCP2	9
Sensor & High-Dimensional	DDG, PE, PS	3
Astronomy & Biology	EW, LS	2

5.7.2 Category-Wise Ranking Summary

Table 5.8 reports the ordinal rank of each method within every domain category, determined by descending mean classification accuracy over the datasets in that group. Rank 1 denotes the best-performing method in the category; rank 4 denotes the worst. The final row identifies the top-ranked model per category.

Table 5.8: Category-wise models ranking summary

Method	Motion & Activity (n=8)	Medical & Neurological (n=9)	Sensor & High-Dim. (n=3)	Astronomy & Biology (n=2)	Overall (n=22)
ST-GoG (ours)	**1st**	**1st**	2nd	3rd	**1st**
SAGoG	2nd	2nd	**1st**	2nd	2nd
ROCKET	3rd	3rd	4th	**1st**	4th
Simple Parallel	4th	4th	3rd	4th	3rd
Top model	ST-GoG	ST-GoG	SAGoG	ROCKET	ST-GoG

5.7.3 Motion and Activity Recognition

ST-GoG ranks first among motion datasets, winning on AWR, HW, LI, and NA. The multi-scale GCN captures structured inter-joint correlations inherent in motion capture data, while the adaptive GoG models temporal gesture phases across the short-to-medium sequence lengths typical of this group.

5.7.4 Medical and Neurological

ST-GoG and SAGoG are statistically indistinguishable in this category. ST-GoG wins on AF, FM, HB, SCP1, and SCP2. The strongest advantage is on AtrialFibrillation (15 training instances), where MixUp augmentation, label smoothing, and seven-pass TTA collectively compensate for extreme data scarcity. SAGoG retains advantages on HMD and MI where the full DTW instance graph encodes class-discriminative inter-instance context.

5.7.5 Sensor and High-Dimensional

SAGoG ranks first, winning on DDG and PS. On DuckDuckGeese ($C = 1345$, $N = 50$), SAGoG's pairwise instance graph substantially outperforms chain-adjacency GoG; data-driven inter-channel correlations are critical when only 50 instances are available.

5.7.6 Astronomy and Biology

ROCKET ranks first, driven by EigenWorms ($T = 17{,}984$). The GoG module creates 10 segments of approximately 1800 time steps each on this dataset, causing computational strain and under-convergence within the 250-epoch budget.

5.8 Ablation Study

Table 5.9: Ablation study on PEMS-SF.

Variant	Best Acc. (%)	Δ vs. Full	Parameters
Full ST-GoG	87.28		1,908,041
w/o Training Aug. Pipeline	87.86	+0.58	1,883,075
w/o SE attention	86.71	−0.57 ↓	1,899,689
w/o Multi-Scale GCN	86.71	−0.57 ↓	1,561,408
w/o GoG module	84.39	−2.89 ↓	1,494,583

Removing the GoG module causes the largest drop (−2.89 pp), confirming hierarchical segment-level relational reasoning as the most critical differentiator of ST-GoG on this dataset. Removing SE attention or the multi-scale GCN each cause a −0.57 pp drop. Disabling the training augmentation pipeline yields a marginal increase on PEMS-SF, consistent with its relatively large training split ($N = 267$); TAP is expected to provide greater benefit on smaller, imbalanced datasets.

5.8.1 Learned Weight Analysis

Table 5.10: Learned fusion weights of the full ST-GoG on PEMS-SF.

Fusion Mechanism	Component	Learned Weight
Multi-Scale GCN	Local ($r = 1$)	0.308
	Medium ($r = 3$)	0.340
	Global ($r = 5$)	0.352
Four-Pathway	TCN global pool	0.285
	TCN + GoG	0.246
	GCN global pool	0.221
	GCN + GoG	0.249

5.9 Discussion of Results

5.9.1 Datasets Where ST-GoG Excels

AtrialFibrillation: With only 15 training instances, ST-GoG achieves 53.33% against SAGoG's 43%. MixUp, seven-pass TTA, and GoG's inter-segment similarity collectively compensate for extreme data scarcity. **Libras:** ST-GoG achieves 95.56% vs. ROCKET's 90.61% on this two-channel, 45-step motion capture dataset; the short sequence falls in the $S = 4$ segment regime and the strong inter-channel coupling makes multi-scale GCN highly effective. **NATOPS:** ST-GoG achieves 99.44% vs. SAGoG's 97.2% on this 24-channel, 51-step aviation gesture dataset, where the hierarchical GoG effectively captures temporal motion phases.

5.9.2 Datasets Where Competitors Are Stronger

FaceDetection: ROCKET (69.42%) and SAGoG (68.3%) outperform ST-GoG (53.04%). With 5890 training instances and 144 channels, per-instance multi-scale GCN and GoG computation creates a training bottleneck within 250 epochs. **EigenWorms:** ROCKET (86.28%) dominates on this $T = 17{,}984$ sequence where the GoG module creates very large inner GCN graphs.

5.9.3 Win/Tie/Loss Summary

Table 5.11: Win/tie/loss count for ST-GoG vs. each baseline across 22 datasets.

Comparison	**Wins**	**Ties**	**Losses**
ST-GoG vs. Baseline TCN-GCN-GoG	20	2	0
ST-GoG vs. SAGoG	14	0	8
ST-GoG vs. ROCKET	16	0	6

5.10 Limitations and Future Improvements

The current implementation carries five limitations. First, the 10-segment GoG on EigenWorms ($T = 17{,}984$) creates inner GCN graphs of roughly 1800 nodes each, making per-epoch computation prohibitive; strided pre-downsampling before the GoG module would mitigate this. Second, on FaceDetection ($N = 5890$), per-instance graph construction is a bottleneck that batch-level graph sharing could resolve. Third, chain adjacency encodes topological proximity rather than data-driven correlations; supplementing with a lightweight cosine-similarity component would improve performance on datasets where cross-channel relationships are class-discriminative but not topologically ordered. Fourth, replacing GoG mean pooling with an attention-weighted readout would allow selective emphasis on the most discriminative temporal segments. Fifth, future evaluations should include F1-scores, per-class confusion matrices, and GFLOPs profiling alongside top-1 accuracy.

6

Conclusions and Future Scope

Accuracy on benchmark test sets is not the only measure of a research contribution. A model that achieves strong performance but requires impractical computation, or whose behaviour depends heavily on dataset-specific hyperparameter search, is a less useful contribution than one that generalises reliably at reasonable cost. The ST-GoG framework was evaluated on both dimensions.

Across 22 UEA datasets, the improved ST-GoG architecture recorded the highest mean test accuracy (0.743) and achieved the most individual-dataset wins relative to any single comparison method, while keeping computational cost well below that of SAGoG's full pairwise instance graph. The advantage was most pronounced on datasets with limited training data such as AtrialFibrillation (15 training instances) and on datasets with structured inter-channel dependencies, including Libras, NATOPS, FingerMovements, and ArticularyWordRecognition, where the combination of multi-scale graph reasoning and data augmentation proved most valuable.

6.1 Summary of Contributions

This thesis presented the **Spatiotemporal Graph-of-Graphs (ST-GoG) Framework** through four cumulative contributions. The **baseline TCN–GCN–GoG model** established a clean, modular pipeline using chain adjacency, confirming that temporal, spatial, and relational encoders can be integrated in a single end-to-end trainable system. The **improved ST-GoG architecture** built on that foundation by adding a five-layer residual TCN with SE attention, a multi-scale GCN operating at three radii ($r \in \{1, 3, 5\}$) with learnable fusion, an adaptive segment-based hierarchical GoG, and a four-pathway learnable softmax fusion head a combination that yields the best average accuracy and ten individual dataset wins across the 22-dataset benchmark. The **advanced training package** MixUp ($\alpha = 0.2$), stochastic depth with exponential decay, cosine annealing with warm restarts, label smoothing, and seven-pass TTA delivered the largest gains on small and imbalanced datasets where the risk of overfitting is greatest. Finally, the **comprehensive empirical study** produced per-dataset accuracy tables, win/tie/loss tallies, average rank scores, and a Wilcoxon-validated critical-difference diagram, providing a thorough and statistically grounded comparison against SAGoG, ROCKET, and the baseline.

6.2 Key Findings

Several consistent empirical patterns emerged from the 22-dataset evaluation. The multi-scale GCN operating at three radii with learnable fusion outperformed the single-radius baseline on datasets with multi-scale temporal dependencies, confirming that spatial modelling at a single fixed scale leaves discriminative inter-channel signals unexploited. SE attention provided consistent but modest gains, most pronounced on datasets where a small subset of temporal feature maps carries disproportionate class-discriminative information. Inspection of learned pathway weights revealed that GoG-augmented streams collectively captured the majority of fusion weight on most datasets, validating the design decision of hierarchical segment-level reasoning over simple global pooling. Despite these advantages, ROCKET remained the strongest competitor on datasets with strong periodic structure or very long sequences, where its random-kernel diversity efficiently extracts discriminative statistics without the computational overhead of graph construction.

6.3 Future Research Directions

6.3.1 Data-Driven Dynamic Adjacency

Extending from chain adjacency to a hybrid approach:

$$A_{hybrid} = (1 - \beta)A_r^{chain} + \beta A^{cosine},$$

where β is a learnable scalar, would allow the model to blend structural topology with data-driven correlations. A fully attention-based adjacency

$$A = \text{Softmax}\left(\frac{(XW_q)(XW_k)^\top}{\sqrt{d}}\right)$$

could capture non-topological relationships and be optimized directly by the classification signal.

6.3.2 Attention-Based GoG Pooling

Replacing mean pooling:

$$\alpha_k = \text{Softmax}(\mathbf{v}^\top \tanh(W\mathbf{h}_k + b)), \quad \mathbf{z}_{GoG} = \sum_k \alpha_k \mathbf{h}_k.$$

6.3.3 Scalable Instance Graph Approximation

For large-N datasets, anchor-based approximations [27] with $m \ll N$ representative anchors reduce memory from $O(N^2)$ to $O(Nm)$.

6.3.4 Temporal Adaptive Sampling

For very long sequences, a learned stride could adaptively downsample before GoG:

$$Z_{sampled} = Z[:, :, :: s], \quad s = f_\theta^{stride}(Z).$$

6.3.5 Interpretability

Adding Grad-CAM [28] visualization for GCN layers and visualizing learned adjacency weights and pathway fusion weights across datasets would improve interpretability for deployment in safety-critical applications.

6.3.6 Pre-Training and Transfer Learning

Large-scale pre-training via masked segment prediction predicting the embedding of a masked temporal segment from surrounding segments could significantly improve performance on extremely small datasets (AF: 15 training instances, SWJ: 12 instances).

6.3.7 Real-Time and Edge Deployment

Optimizing ST-GoG for edge devices through knowledge distillation, INT8 quantization, and pruning of the multi-scale GCN to a sparser single-scale GCN at inference time would enable deployment in latency-constrained environments such as real-time arrhythmia monitoring.

A

Additional Experimental Details

A.1 Dataset Descriptions by Domain

The UEA Multivariate Time Series Archive [10] contains datasets from diverse domains. The **Motion and Activity Recognition** group (AWR, HW, LI, NA, RS, SWJ, UGL) comprises multi-sensor inertial and position recordings in which discriminative patterns reside in the phase and relative dynamics between channels. The **Medical and Neurological** group (AF, EP, HB, SCP1, SCP2, HMD, FM, FD, MI) covers electrocardiographic and electroencephalographic recordings characterised by small training cohorts and clinically meaningful inter-class distinctions. The **Sensor Networks and Traffic** group is represented principally by PEMS-SF, which aggregates readings from 267 road-traffic-flow sensors over 144 time steps and exhibits strong daily periodicity. The **Astronomy and Other** group includes LSST, whose six-channel photometric light curves capture stellar variability, as well as DuckDuckGeese, which encodes bird wing-motion dynamics, and EigenWorms, which represents nematode body-shape evolution via six eigenworm coefficients sampled at 17,984 time steps.

A.2 Full Hyperparameter Settings

Table A.1: Full hyperparameter settings for both baseline and improved model

Hyperparameter	**Baseline**	**ST-GoG (Improved)**
Learning rate	10^{-3}	10^{-3}
LR schedule	None	CosineAnnealingWarmRestarts
T_0 (restart period)	–	50
T_{mult}	–	2
η_{min}	–	10^{-6}
Weight decay	10^{-4}	10^{-4}
Optimizer	AdamW	AdamW
Gradient clipping	–	1.0 (max norm)
Batch size	8–32	8–32
Epochs	50–100	250
Early stopping	patience 15	best accuracy tracked
TCN dilation	$\{1, 2, 4\}$	$\{1, 2, 4, 8, 16\}$
TCN filters F	64	128
GCN hidden dim	64	128
GCN layers	2	3 per scale
GCN radii	3 (single chain)	$\{1, 3, 5\}$ (multi-scale chain)
SE reduction ratio	–	4
GoG segments S	–	4/6/8/10 (adaptive)
GoG embedding	128	128
Dropout	0.1	0.42
Label smoothing ϵ	–	0.10 ($K \leq 5$), 0.15 ($K > 5$)
MixUp α	–	0.2
Stochastic depth	–	$0.2 \cdot e^{-t/60}$
TTA N_{aug}	–	7
TTA σ	–	0.003
Reproducibility seed	–	42
Mixed precision	–	autocast + GradScaler

A.3 Ablation Study Design

A systematic ablation study is recommended for future work:

Table A.2: Proposed ablation study variants

Variant	**Removed/Changed Component**
ST-GoG (full)	None (full model)
w/o SE attention	SE blocks removed from ResTCN
w/o Multi-scale GCN	Single-radius GCN ($r = 3$) only
w/o GoG	Mean pooling of TCN/GCN outputs only
w/o MixUp	MixUp augmentation disabled
w/o Label Smoothing	Standard cross-entropy
w/o Pathway Fusion	Fixed equal weights ($\alpha_i = 0.25$)
w/o TTA	Single forward pass at inference
Chain vs. Cosine Adj.	Cosine-similarity adjacency instead of chain
Adaptive vs. Fixed Seg.	Fixed $S = 4$ regardless of T

A.4 Project Timeline

Table A.3: Work Plan and Timeline

Phase	**Period**	**Focus Area**	**Key Outcomes**
Phase 1	Oct – Nov 2025	Research groundwork	Literature review, gap analysis, dataset selection, research objectives
Phase 2	Dec 2025 – Jan 2026	Baseline development	TCN–GCN–GoG implementation with chain adjacency, baseline experiments
Phase 3	Jan – Feb 2026	Improved architecture	SE attention, multi-scale GCN, adaptive GoG, MixUp, TTA, label smoothing
Phase 4	Feb – Mar 2026	Evaluation and writing	Full 22-dataset benchmark, critical difference analysis, thesis writing

Bibliography

[1] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[2] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[3] F. Gao, L. Zhang, and Y. Zhang, "FDNet: A novel multivariate time series classification model through fusing feature and difference," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2024.

[4] S. Wang, Y. Zhang, X. Lin, Y. Hu, Q. Huang, and B. Yin, "SAGoG: Similarity-Aware Graph of Graphs Neural Networks for Multivariate Time Series Classification," *IEEE Transactions on Knowledge and Data Engineering*, 2025, early access.

[5] P. Schäfer and U. Leser, "Multivariate time series classification with WEASEL+MUSE," *arXiv preprint arXiv:1711.11343*, 2017.

[6] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, pp. 1454–1495, 2020.

[7] A. Dempster, D. F. Schmidt, and G. I. Webb, "MiniRocket: A very fast (almost) deterministic transform for time series classification," in *Proc. ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2021.

[8] H. I. Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean, "InceptionTime: Finding AlexNet for time series classification," *Data Mining and Knowledge Discovery*, vol. 34, pp. 1936–1962, 2020.

[9] J. Zhang, Y. Zhu, X. Chen, S. Liu, and J. Song, "TD-GCN: Temporal Dynamic Graph Convolutional Network for Multivariate Time Series Classification," *Pattern Recognition Letters*, 2023.

[10] A. Bagnall, H. A. Dau, J. Lines, M. Flynn, J. Large, A. Bostrom, P. Southam, and E. Keogh, "The UEA multivariate time series classification archive, 2018," *arXiv preprint arXiv:1811.00075*, 2018. [Online]. Available: `http://www.timeseriesclassification.com/`

[11] D. J. Berndt and J. Clifford, "Using dynamic time warping to find patterns in time series," in *Proc. AAAI Workshop Knowledge Discovery in Databases*, 1994.

[12] Z. Zhang, Y. Zhao, H. Liu, and Y. Chen, "TapNet: Multivariate time series classification with attentional prototypical network," in *Proc. AAAI Conf. Artificial Intelligence (AAAI)*, 2020.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[15] Z. Wang, W. Yan, and T. Oates, "Time series classification from scratch with deep neural networks: A strong baseline," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2017.

[16] F. Karim, S. Majumdar, H. Darabi, and S. Chen, "LSTM fully convolutional networks for time series classification," *IEEE Access*, vol. 6, pp. 1662–1669, 2018.

[17] I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: A review," *Data Mining and Knowledge Discovery*, vol. 33, pp. 917–963, 2019.

[18] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive study on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.

[19] J. Lines and A. Bagnall, "Time series classification with ensembles of elastic distance measures," *Data Mining and Knowledge Discovery*, vol. 29, pp. 565–592, 2015.

[20] J. Lin, E. Keogh, L. Wei, and S. Lonardi, "Experiencing SAX: A novel symbolic representation of time series," *Data Mining and Knowledge Discovery*, vol. 15, pp. 107–144, 2007.

[21] P. Schäfer, "The BOSS is concerned with time series classification in the presence of noise," *Data Mining and Knowledge Discovery*, vol. 29, pp. 1505–1530, 2015.

[22] A. Shokoohi-Yekta, B. Hu, H. Jin, J. Wang, and E. Keogh, "Generalizing DTW to the multi-dimensional case requires an adaptive approach," *Data Mining and Knowledge Discovery*, vol. 31, pp. 1–31, 2017.

[23] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "Mixup: Beyond empirical risk minimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[24] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[25] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[26] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artificial Intelligence (AAAI)*, 2021.

[27] W. Liu, J. He, and S.-F. Chang, "Large graph construction for scalable semi-supervised learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2010.

[28] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2017.

[29] M. Vlachos, G. Kollios, and D. Gunopulos, "Discovering similar multidimensional trajectories," in *Proc. IEEE Int. Conf. Data Engineering (ICDE)*, 2002.

[30] L. Chen and R. Ng, "On the marriage of Lp-norms and edit distance," in *Proc. VLDB Endowment*, 2004.

[31] P.-F. Marteau, "Time warp edit distance with stiffness adjustment for time series matching," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 2, pp. 306–318, 2009.

[32] J. Hills, J. Lines, E. Baranauskas, J. Mapp, and A. Bagnall, "Classification of time series by shapelet transformation," *Data Mining and Knowledge Discovery*, vol. 28, pp. 851–881, 2014.